BioPathNet++ for Autoimmune Drug Repurposing: Complete Project Overview

📋 Table of Contents

1. Problem Statement
2. Why This Matters
3. Our Solution
4. Technical Approach
5. Dataset & Data Preparation
6. Model Architecture
7. Implementation Plan
8. Expected Results
9. Validation Strategy
10. Team & Timeline

---

1. Problem Statement

What is Drug Repurposing?

Drug repurposing (also called drug repositioning) is the process of finding **new uses for existing, approved drugs**. Instead of developing a brand new drug from scratch (which costs $2.6 billion and takes 10-15 years), we identify existing drugs that could treat different diseases.

Example:

- Aspirin was originally for pain relief
- Later discovered to prevent heart attacks
- This is drug repurposing!

The Specific Problem We're Solving

Target Diseases: Rare autoimmune diseases with limited treatment options

- Sjögren's Syndrome (only 14 known drug associations!)
- Systemic Sclerosis (173 associations)
- Primary Biliary Cholangitis (207 associations)
- Myasthenia Gravis (109 associations)
- Autoimmune Hepatitis (56 associations)

Why These Diseases?

1. **Rare** - Affect small patient populations (orphan diseases)
2. **Under-researched** - Pharmaceutical companies don't invest (no profit incentive)
3. **Limited data** - Very few known drug-disease associations
4. **Shared mechanisms** - All are autoimmune (immune system attacks own body)

The Challenge: Traditional machine learning requires **lots of training data**. For rare diseases, we have almost none. How do we predict effective drugs when we barely have any examples?

---

2. Why This Matters

Medical Impact

For Patients:

- Faster access to treatments (years vs decades)
- Lower costs (repurposed drugs are already approved)
- Better quality of life for rare disease patients
- Potentially life-saving for conditions with no current treatments

For Healthcare:

- Reduced drug development costs
- Faster clinical trials (safety already established)
- More treatment options for physicians

Real Numbers:

- Traditional drug development: **$2.6 billion, 10-15 years**
- Drug repurposing: **$300 million, 3-5 years**
- Success rate: **30% vs 10%** for new drugs

Scientific Impact

Knowledge Graph Medicine:

- Integrates diverse biomedical data sources
- Reveals hidden connections between drugs and diseases
- Enables computational hypothesis generation

AI in Healthcare:

- Demonstrates transfer learning in biomedicine
- Shows path-based reasoning for interpretable predictions

- Advances precision medicine

---

3. Our Solution

Core Idea: Transfer Learning

The Analogy: Imagine you learned to play the guitar. Now someone asks you to play the ukulele (which you've never touched). You won't be a complete beginner because:

- Both have strings
- Both use similar chord patterns
- Your guitar knowledge **transfers** to ukulele

Our Approach:

1. **Train** on data-rich autoimmune diseases (Rheumatoid Arthritis, Lupus, Multiple Sclerosis)
2. **Learn** common autoimmune mechanisms and treatment patterns
3. **Transfer** this knowledge to rare diseases (Sjögren's Syndrome) with minimal data

Why Graph Neural Networks?

Knowledge Graphs: A knowledge graph represents biomedical knowledge as:

- **Nodes** (entities): Drugs, Diseases, Genes, Proteins, Pathways
- **Edges** (relationships): "Drug treats Disease", "Gene causes Disease", "Protein interacts with Protein"

Example:

The Insight: If Methotrexate treats Rheumatoid Arthritis, and both RA and Sjögren's involve TNF-α and IL-6, then maybe Methotrexate could treat Sjögren's too!

Graph Neural Networks (GNNs):

- Learn patterns from graph structure
- Capture multi-hop relationships (drug → gene → pathway → disease)
- Generalize to new, unseen diseases

---

4. Technical Approach

4.1. The Data Sources

We combine **three major biomedical databases**:

DRKG (Drug Repurposing Knowledge Graph)

- **Size:** 5,874,261 relationships (triples)
- **Entities:** 97,238 (drugs, diseases, genes, proteins, pathways, etc.)
- **What it contains:**
  - Drug-disease associations (which drugs treat which diseases)
  - Drug-gene interactions (which genes drugs target)
  - Gene-disease associations (which genes cause diseases)
  - Protein-protein interactions
  - Pathway information

Example Triple:

(Methotrexate, treats, Rheumatoid Arthritis)
(TNF, associated_with, Rheumatoid Arthritis)
(Methotrexate, targets, DHFR)

Orphanet (Rare Disease Database)

- **Size:** ~115,878 disease-phenotype associations
- **What it contains:**
  - Rare disease classifications
  - Disease-gene associations (which genes cause rare diseases)
  - Disease-phenotype mappings (what symptoms rare diseases cause)

Why We Need It: DRKG has limited data on rare diseases. Orphanet fills this gap with curated rare disease information.

HPO (Human Phenotype Ontology)

- **Size:** ~603,246 annotations
- **What it contains:**

- Disease-phenotype associations (diseases → symptoms)
- Gene-phenotype associations (genes → symptoms)
- Standardized phenotype terminology

Why We Need It: Connects diseases through shared symptoms. If two diseases have similar phenotypes, they might respond to similar treatments.

4.2. Data Enrichment Strategy

Problem:

- Sjögren's Syndrome has only **14 drug associations** in DRKG
- Not enough data to train a model!

Solution: Enrich the graph by adding data from Orphanet and HPO:

Before Enrichment (DRKG only):

```
Sjögren's Syndrome: 14 links (5 drugs, 9 genes)
```

After Enrichment (DRKG + Orphanet + HPO):

```
Sjögren's Syndrome: ~99 links
  - Direct drug links: 14
  - New gene associations: +35 (from Orphanet)
  - New phenotype links: +50 (from HPO)

Improvement: +607% coverage!
```

How Enrichment Helps: Even though we don't have more direct drug-Sjögren's links, we now have:

- More genes associated with Sjögren's
- Phenotype similarities with other autoimmune diseases
- Indirect pathways: Drug → Gene → Phenotype → Sjögren's

4.3. The Nine Target Diseases

We strategically selected 9 autoimmune diseases:

Training Set (Data-Rich - Learn From These):

Disease	Links	Drug Links	Gene Links	Role
Rheumatoid Arthritis	1,657	395	1,161	Primary training
Systemic Lupus Erythematosus	629	173	384	Training
Multiple Sclerosis	655	193	345	Training

Why These?

- Well-studied diseases
- Lots of known drugs and mechanisms
- Model learns general autoimmune treatment patterns

Target Set (Data-Sparse - Transfer Knowledge Here):

Disease	Links	Drug Links	Gene Links	Challenge
Sjögren's Syndrome ⭑	14	5	9	**Zero-shot** (almost no data!)
Crohn's Disease	123	59	64	Few-shot
Myasthenia Gravis	109	70	39	Few-shot
Primary Biliary Cholangitis	200	61	116	Few-shot
Systemic Sclerosis	141	62	79	Few-shot
Autoimmune Hepatitis	51	32	19	Few-shot

Why These?

- Rare diseases with limited treatment options
- Share autoimmune mechanisms with training diseases
- Perfect test cases for transfer learning

---

5. Dataset & Data Preparation

5.1. What We Actually Did (Phase 1)

Week 1: Dataset Download & Verification

Downloaded:

1. **DRKG:** 5.8M triples (1.2 GB file)
2. **Orphanet:** 3 XML files (~100 MB)
    - Disease-gene associations
    - Disease-phenotype associations
    - Disease nomenclature
3. **HPO:** 3 files (~30 MB)
    - Phenotype annotations
    - HPO ontology
    - Gene-phenotype mappings

Verified:

- File integrity (no corruption)
- Correct formats (TSV, XML)
- Expected sizes and triple counts

Week 2: Data Extraction & Processing

Step 1: Extract Autoimmune Subgraph

Problem: DRKG has 5.8M triples covering ALL diseases (cancer, diabetes, heart disease, etc.). We only need autoimmune diseases.

Solution: Extract autoimmune-specific subgraph

```
Original DRKG:          5,874,261 triples
Autoimmune Subgraph:    1,245,487 triples
Reduction:              78.8% (focused dataset)
```

How We Did It:

1. Identify our 9 diseases using MESH IDs
    - Example: Sjögren's = `Disease::MESH:D012859`
2. Extract all edges directly connected to these diseases
3. Expand to 2-hop neighborhood (drugs → genes → diseases)
4. Keep only relevant entities and relationships

What's in the Subgraph:

- 54,985 entities
    - 23,279 drugs (compounds)
    - 18,922 genes
    - 950 diseases (including related conditions)
    - 11,381 biological processes
    - 1,822 pathways
- 107 relation types
    - Drug treats Disease
    - Gene associated_with Disease
    - Gene interacts Gene
    - Protein regulates Pathway
    - etc.

Step 2: Parse Orphanet XML Files

Extracted from Orphanet:

```
Disease-Phenotype associations: 115,878 triples
Format: (Disease::ORPHA:12894, has_phenotype, Phenotype::HP:0001234)
```

Example:

```
Sjögren's Syndrome → Dry eyes (HP:0000238)
Sjögren's Syndrome → Dry mouth (HP:0000217)
Sjögren's Syndrome → Fatigue (HP:0012378)
```

Step 3: Parse HPO Annotations

Extracted from HPO:

```
Disease-Phenotype: 280,731 triples
Gene-Phenotype:    322,515 triples
Total:             603,246 triples
```

Why This Matters: Connects diseases through shared phenotypes and genes. Even if Drug X doesn't directly link to Sjögren's, if it treats symptoms that Sjögren's causes, it's a candidate!

Step 4: Merge All Datasets

Combined everything:

```
DRKG Subgraph:      1,245,487 triples
Orphanet:             115,878 triples
HPO:                  603,246 triples
─────────────────────────────────────
Total (after dedup): 1,906,058 triples

Increase: +53.0% more knowledge!
```

Final Enriched Graph:

- 84,712 unique entities
- 2,435 unique relation types
- 1,906,058 triples

5.2. Entity Embeddings (BioBERT)

What are Embeddings?

Computers can't understand text like "Methotrexate" or "Rheumatoid Arthritis". We need to convert entities into numbers (vectors) that capture their meaning.

Example:

```
"Methotrexate" → [0.23, -0.45, 0.67, ..., 0.12]  (768 numbers)
                  ↑
              Embedding vector
```

Similar drugs have similar vectors:

Methotrexate: [0.23, -0.45, 0.67, ...]
Azathioprine: [0.21, -0.43, 0.69, ...] ← Very similar!
Aspirin: [0.89, 0.12, -0.34, ...] ← Very different

BioBERT:

- A language model trained on biomedical text (PubMed papers)
- Understands medical terminology
- Generates 768-dimensional embeddings

What We Did:

python
For each of our 84,712 entities:
1. Extract entity name/ID
2. Pass through BioBERT
3. Get 768-dim vector
4. Save to file

Output: entity_embeddings.npy (248 MB)
      Shape: (84,712 entities × 768 dimensions)
```

**Why This Matters:**
These embeddings serve as **initial node features** for our graph neural network. The model starts with meaningful representations of each entity.

### **5.3. Background Regulatory Graph (BRG)**

**What is BRG?**

A subgraph containing **immune-specific regulatory relationships**:
- Gene-gene interactions (protein binding, regulation)
- Immune pathways (JAK-STAT, NF-κB, TNF signaling)
- Cytokine cascades (IL-6, IL-17, TNF-α networks)

**Why We Need It:**

Regular knowledge graphs show "Drug treats Disease" but don't explain **WHY**. The BRG provides biological context.

**Example Without BRG:**
```
Methotrexate --[treats]--> Rheumatoid Arthritis
                    (How? Why? Unknown!)
```

**Example With BRG:**
```
Methotrexate --[inhibits]--> DHFR (gene)
                                ↓
                    Purine synthesis ↓
                                ↓
                    T-cell proliferation ↓

↓
TNF-α production ↓
↓
Inflammation ↓
↓
Rheumatoid Arthritis ↓
```

**What We Extracted:**
```
BRG Statistics:
  Edges: 567,959 regulatory relationships
  Entities: 23,971 (mostly genes)
  Relation types: 66

Key Components:
  - Gene-gene interactions: 552,380
  - Immune-relevant edges: 15,579
  - Pathway edges: 14,175
```

**How We Built It:**
1. Extract all gene-gene interactions
2. Filter for immune-relevant keywords:
   - Cytokines: TNF, IL-, IFN
   - Signaling: JAK, STAT, NF-κB, MAPK
   - Immune cells: CD4, CD8, CD20
   - Processes: inflammation, immune response
3. Include pathway relationships
4. Merge and deduplicate

### **5.4. PyTorch Geometric Data**

**What is PyTorch Geometric (PyG)?**

A library for Graph Neural Networks. We need to convert our knowledge graph into a format PyG can process.

**Conversion:**

**Original Format (TSV):**
```
Methotrexate   treats   Rheumatoid_Arthritis
TNF   associated_with   Rheumatoid_Arthritis
```

## PyG Format:

python
```
Data(
   x = tensor([[0.23, -0.45, ...], # Node 0: Methotrexate embedding
               [0.67, 0.12, ...],   # Node 1: RA embedding
               ...]),   # 84,712 nodes total

   edge_index = tensor([[0, 1],   # Edge: Node 0 → Node 1
                        [1, 0],   # Edge: Node 1 → Node 0 (bidirectional)
                        ...]),   # 1,906,058 edges total
```
```

```
    edge_type = tensor([15,          # Edge type 15 = "treats"
                        89,          # Edge type 89 = "associated_with"
                        ...])        # 2,435 edge types
)
```

Train/Val/Test Splits:

We created disease-based splits (not random):

Training Set: Drug-disease pairs from data-rich diseases
```
Rheumatoid Arthritis pairs:  ~2,500 examples
SLE pairs:                   ~1,000 examples
MS pairs:                    ~1,000 examples
─────────────────────────────────────────────
Total training:              ~3,500 examples (80%)
Validation:                  ~900 examples (20%)
```

Test Set (Zero-Shot): Drug-disease pairs from data-sparse diseases
```
Sjögren's pairs:             ~14 examples  ⭐ (main test case!)
Crohn's pairs:               ~123 examples
Myasthenia pairs:            ~109 examples
Others:                      ~300 examples
─────────────────────────────────────────────
Total test:                  ~546 examples
```

Why This Split?

Traditional ML: Random 80/20 split
```
Problem: Leakage! Model sees some Sjögren's examples during training
```

Our approach: Disease-based split
```
Training: Only see RA, SLE, MS
Testing: Only see Sjögren's, Crohn's, etc. (never seen before!)
This is TRUE transfer learning!
```

---

6. Model Architecture

6.1. BioPathNet++ Overview

BioPathNet++ = NBFNet + Metapaths + BRG + Transfer Learning

Think of it as a 3-stage pipeline:
```
Stage 1: Path Finding (NBFNet)
```

↓
Stage 2: Path Weighting (Attention)
↓
Stage 3: Prediction (Scoring)
```

### **6.2. What are Metapaths?**

**Metapath:** A specific pattern of relationships in the graph.

**Example Metapaths for Drug Repurposing:**

**1. Drug → Gene → Disease**
```
Methotrexate --[targets]--> DHFR --[associated_with]--> RA
```
*Interpretation: Methotrexate targets DHFR gene, which is involved in RA*

**2. Drug → Gene → Gene → Disease**
```
Methotrexate --[targets]--> DHFR --[interacts]--> TNF --[causes]--> RA
```
*Interpretation: Methotrexate affects DHFR, which interacts with TNF, which causes RA*

**3. Drug → Pathway → Disease**
```
Methotrexate --[inhibits]--> Purine_Synthesis --[dysregulated_in]--> RA
```
*Interpretation: Methotrexate inhibits a pathway dysregulated in RA*

**4. Drug → Phenotype → Disease**
```
Methotrexate --[treats]--> Joint_Pain --[symptom_of]--> Sjögren's
```
*Interpretation: Methotrexate treats joint pain, which is a symptom of Sjögren's*

**We Define 7 Metapaths:**
1. Drug → Gene → Disease
2. Drug → Gene → Gene → Disease
3. Drug → Pathway → Disease
4. Drug → Protein → Disease
5. Drug → Phenotype → Disease
6. Drug → Disease → Disease (similar diseases)
7. Drug → Gene → Pathway → Disease

**Why Multiple Metapaths?**

Different metapaths capture different biological mechanisms. The model learns which paths are most predictive for each disease.

### **6.3. NBFNet (Neural Bellman-Ford Network)**

**What is Bellman-Ford?**

An algorithm that finds shortest paths in a graph. NBFNet adapts this for neural learning.
```

Traditional Graph Neural Networks (like GCN):
```
Problem: Only look at immediate neighbors
         Can't capture long-range dependencies
```

NBFNet:
```
Solution: Explicitly models paths from source to target
          Learns path-based reasoning
```

How NBFNet Works:

Step 1: Initialize
```
Start node (Drug): Embedding = BioBERT vector
All other nodes: Embedding = zeros
```

Step 2: Message Passing (K iterations)
```
For each iteration k = 1 to K:
    For each node v:
        1. Gather messages from all neighbors
        2. Weight messages by edge types
        3. Update node embedding

This is like Bellman-Ford: each iteration extends paths by one hop
```

Step 3: Path Extraction
```
After K iterations, each node has accumulated information
about all paths of length ≤ K from the start node
```

Mathematically:
```
h_v^(k) = σ(∑_{u ∈ N(v)} W_r^(k) · [h_u^(k-1) || e_r])

Where:
  h_v^(k) = embedding of node v at iteration k
  N(v) = neighbors of node v
  W_r^(k) = learnable weight for relation type r
  e_r = relation embedding
  || = concatenation
  σ = activation function (ReLU)
```

Intuition: Each node "gathers" information from its neighbors, weighted by the relation type. After K iterations, a node knows about all paths of length K.

6.4. Complete BioPathNet++ Architecture

python

```python
class BioPathNetPlusPlus(nn.Module):
    def __init__(self,
                 num_entities,
                 num_relations,
                 embedding_dim=768,
                 num_layers=3,
                 num_metapaths=7):

        # 1. Entity Embeddings (initialized with BioBERT)
        self.entity_embeddings = nn.Embedding(
            num_entities,
            embedding_dim
        )
        # Load BioBERT embeddings as initialization
        self.entity_embeddings.weight.data = load_biobert_embeddings()

        # 2. Relation Embeddings (learned)
        self.relation_embeddings = nn.Embedding(
            num_relations,
            embedding_dim
        )

        # 3. NBFNet Layers (path encoder)
        self.nbfnet_layers = nn.ModuleList([
            NBFNetConv(embedding_dim, num_relations)
            for _ in range(num_layers)
        ])

        # 4. Metapath Attention
        self.metapath_attention = nn.MultiheadAttention(
            embedding_dim,
            num_heads=8
        )

        # 5. BRG Encoder (regulatory context)
        self.brg_encoder = nn.GRU(
            input_size=embedding_dim,
            hidden_size=embedding_dim,
            num_layers=2
        )

        # 6. Prediction Head
        self.scorer = nn.Sequential(
            nn.Linear(embedding_dim * 2, embedding_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(embedding_dim, 1)
        )

    def forward(self, query_drug, query_relation, candidate_diseases):
        """
        Args:
            query_drug: Drug entity ID (e.g., Methotrexate)
            query_relation: Relation type (e.g., "treats")
            candidate_diseases: List of disease IDs to score
```

```
        Returns:
            scores: Probability for each candidate disease
        """

        # Step 1: Encode paths using NBFNet
        drug_emb = self.entity_embeddings(query_drug)
        rel_emb = self.relation_embeddings(query_relation)

        # NBFNet message passing (K iterations)
        h = drug_emb
        for layer in self.nbfnet_layers:
            h = layer(h, self.graph, rel_emb)

        # Now h contains path information from drug to all entities

        # Step 2: Extract metapaths
        metapaths = self.extract_metapaths(
            drug_emb,
            candidate_diseases,
            h
        )
        # metapaths shape: (num_metapaths, embedding_dim)

        # Step 3: Attention over metapaths
        weighted_paths, attention_weights = self.metapath_attention(
            metapaths,
            metapaths,
            metapaths
        )
        # Learns which metapaths are important for this drug-disease pair

        # Step 4: Incorporate BRG context
        brg_context = self.get_brg_context(
            query_drug,
            candidate_diseases
        )
        regulatory_info, _ = self.brg_encoder(brg_context)

        # Step 5: Combine and score
        combined = torch.cat([weighted_paths, regulatory_info], dim=-1)
        scores = self.scorer(combined)

        return scores, attention_weights
```

What Each Component Does:

1. Entity Embeddings:
- Initialize with BioBERT vectors
- Fine-tune during training
- Captures semantic meaning of drugs, diseases, genes

2. NBFNet Layers:
- Learns to traverse the graph
- Encodes multi-hop paths

- Captures complex drug-disease relationships

3. Metapath Attention:
- Scores different path types
- Learns which biological mechanisms matter
- Example: For RA, Drug→Gene→Disease might be important
  For Sjögren's, Drug→Phenotype→Disease might be key

4. BRG Encoder:
- Incorporates immune-specific knowledge
- Provides mechanistic context
- Uses GRU to model regulatory cascades

5. Prediction Head:
- Combines all information
- Outputs probability score
- Higher score = more likely the drug treats the disease

6.5. Transfer Learning Strategy

The Key Innovation:

Pre-training Phase:
```
Train on: RA + SLE + MS (data-rich)
Learn: General autoimmune treatment patterns
```

Transfer Phase:
```
Fine-tune on: Sjögren's (data-sparse)
Use: Only 5-10 known examples (few-shot)
OR
Use: Zero examples (zero-shot)
```

How Transfer Works:

1. Shared Mechanisms:
```
All autoimmune diseases involve:
  - Immune dysregulation
  - Inflammation
  - Cytokine cascades (TNF-α, IL-6, IL-17)
  - Autoantibody production

Model learns these patterns from RA/SLE/MS,
then applies them to Sjögren's
```

2. Shared Genes:
```
RA and Sjögren's both involve:
  - HLA genes (MHC)
  - TNF pathway
  - B-cell activation
```

Drugs targeting these in RA likely work for Sjögren's
```

**3. Shared Phenotypes:**
```
RA symptoms: Joint pain, fatigue, inflammation
Sjögren's symptoms: Joint pain, fatigue, dry eyes

Drugs treating RA's joint pain might help Sjögren's joint pain
```

**Training Procedure:**

**Phase 1: Pre-train (Data-Rich Diseases)**

```
python
for epoch in range(num_epochs):
    for batch in train_loader:  # RA, SLE, MS pairs
        drug, disease, label = batch

        # Forward pass
        score = model(drug, "treats", disease)

        # Loss
        loss = binary_cross_entropy(score, label)

        # Backward
        loss.backward()
        optimizer.step()
```

Phase 2: Transfer (Data-Sparse Diseases)

```
python
# Option A: Zero-shot (no fine-tuning)
# Just use the pre-trained model directly!
scores = model(new_drug, "treats", sjogrens)

# Option B: Few-shot (fine-tune on 5-10 examples)
for epoch in range(few_epochs):
    for drug, disease, label in few_shot_data:  # 5-10 Sjögren's examples
        score = model(drug, "treats", disease)
        loss = binary_cross_entropy(score, label)
        loss.backward()
        optimizer.step()
```

Why This Works:

The model learns **abstract autoimmune patterns** rather than disease-specific rules. These patterns generalize to unseen diseases.

Example:
```
Pattern learned from RA:
```
```

"If drug inhibits TNF-α, it treats autoimmune disease"

Applied to Sjögren's:
Model sees: Drug X inhibits TNF-α
Model infers: Drug X might treat Sjögren's (even though it never saw this specific drug-Sjögren's pair during training)
```

---

## **7. Implementation Plan**

### **Phase 1: Data Preparation (Weeks 1-3) ✅ COMPLETE**

**Week 1 (Feb 3-9):** ✅
- Download DRKG, Orphanet, HPO
- Verify data integrity
- Identify 9 target diseases

**Week 2 (Feb 10-16):** ✅
- Extract autoimmune subgraph (1.2M triples)
- Parse Orphanet XML (116K triples)
- Parse HPO annotations (603K triples)
- Merge datasets (1.9M triples total)

**Week 3 (Feb 17-23):** ✅
- Generate BioBERT embeddings (84K entities)
- Build Background Regulatory Graph (568K edges)
- Create PyTorch Geometric data
- Define train/val/test splits

**Deliverables:** ✅
- `enriched_drkg.tsv` - 1.9M triple knowledge graph
- `entity_embeddings.npy` - 84K × 768 embeddings
- `pyg_graph.pt` - PyG data object
- `train_pairs.csv`, `val_pairs.csv`, `test_pairs.csv`

### **Phase 2: Model Implementation (Weeks 4-9) 🔄 NEXT**

**Week 4 (Feb 24 - Mar 2): NBFNet Implementation**
- Implement NBFNetConv layer
- Test message passing on toy graph
- Verify gradient flow
- **Deliverable:** Working NBFNet encoder

**Week 5 (Mar 3-9): Metapath Definition & Extraction**
- Define 7 metapaths
- Implement metapath extraction
- Test on sample drug-disease pairs
- **Deliverable:** Metapath extractor module

**Week 6 (Mar 10-16): BRG Integration**
- Implement BRG encoder (GRU)
- Extract regulatory paths
- Combine with NBFNet features
```

- **Deliverable:** Complete encoder (NBFNet + BRG)

Week 7 (Mar 17-23): Attention & Prediction Head
- Implement metapath attention
- Build prediction head
- End-to-end forward pass
- **Deliverable:** Complete BioPathNet++ model

Week 8 (Mar 24-30): Training Pipeline
- Implement training loop
- Add logging (TensorBoard/Weights & Biases)
- Set up checkpointing
- **Deliverable:** Training infrastructure

Week 9 (Mar 31 - Apr 6): Pre-training
- Train on RA/SLE/MS (data-rich)
- Hyperparameter tuning
- Monitor convergence
- **Deliverable:** Pre-trained model checkpoint

Phase 3: Transfer Learning & Evaluation (Weeks 10-15)

Week 10 (Apr 7-13): Transfer to Rare Diseases
- Zero-shot evaluation on Sjögren's
- Few-shot fine-tuning experiments
- Compare zero-shot vs few-shot
- **Deliverable:** Transfer learning results

Week 11 (Apr 14-20): Validation - Layer 1 (RepoDB)
- Test against RepoDB gold standard
- Calculate AUPRC, Hit@K metrics
- Compare to baselines
- **Deliverable:** RepoDB validation report

Week 12 (Apr 21-27): Validation - Layers 2-3
- Layer 2: Literature mining validation
- Layer 3: Clinical trial evidence
- Generate validation report
- **Deliverable:** Multi-layer validation results

Week 13 (Apr 28 - May 4): Validation - Layers 4-5
- Layer 4: Pathway analysis
- Layer 5: Expert review preparation
- Compile top predictions
- **Deliverable:** Complete validation report

Week 14 (May 5-11): Analysis & Interpretation
- Analyze attention weights (which metapaths matter?)
- Case studies: Top drug candidates for Sjögren's
- Mechanistic interpretation
- **Deliverable:** Analysis report with interpretations

Week 15 (May 12-18): Final Presentation
- Prepare slides
- Create visualizations
- Rehearse presentation

- **Deliverable:** Final presentation & report

---

8. Expected Results

8.1. Performance Metrics

Primary Metrics:

1. AUPRC (Area Under Precision-Recall Curve)
```
Why AUPRC, not AUROC?
  - Drug repurposing is highly imbalanced
  - Most drug-disease pairs are negative (drug doesn't treat disease)
  - AUPRC better captures performance on rare positives
```

Expected Performance:

On RepoDB (Gold Standard):
```
Baseline (TransE):          AUPRC = 0.65
TxGNN (SOTA):               AUPRC = 0.92
BioPathNet++ (Our Target):  AUPRC = 0.92-0.95

Goal: Match or exceed SOTA!
```

On Sjögren's (Zero-Shot):
```
Random:                     AUPRC = 0.01 (1% of drugs work)
Our Target:                 AUPRC = 0.80-0.85

This would be GROUNDBREAKING for a zero-shot model!
```

2. Hit@K (Top-K Accuracy)
```
Hit@10:  % of true drugs in top 10 predictions
Hit@50:  % of true drugs in top 50 predictions
Hit@100: % of true drugs in top 100 predictions

Target:
  Hit@10 ≥ 60%  (6 out of 10 predictions are correct)
  Hit@50 ≥ 80%  (40 out of 50 predictions are correct)
```

3. Transfer Learning Metrics

Transfer Hit Rate:
```
% of drugs that treat data-rich disease
AND predicted to treat data-sparse disease
AND actually work
```

Target: 60-70% transfer hit rate
```

**Knowledge Transfer Score:**
```
How much knowledge from RA/SLE/MS transfers to Sjögren's?

Measured by:
  - Shared predicted mechanisms
  - Overlapping metapaths
  - Common regulatory pathways

Target: 70%+ shared mechanisms
```

### **8.2. Example Predictions for Sjögren's**

**Top 10 Predicted Drugs (Hypothetical):**

| Rank | Drug | Mechanism | Confidence | Evidence |
|---|---|---|---|---|
| 1 | Rituximab | B-cell depletion | 0.94 | ✓ Clinical trials |
| 2 | Hydroxychloroquine | Immune modulation | 0.89 | ✓ Literature |
| 3 | Methotrexate | TNF-α inhibition | 0.87 | ✓ Off-label use |
| 4 | Tocilizumab | IL-6 blockade | 0.85 | ? Novel |
| 5 | Abatacept | T-cell co-stimulation | 0.83 | ✓ Case reports |
| 6 | Belimumab | B-cell survival inhibition | 0.81 | ? Novel |
| 7 | Leflunomide | Pyrimidine synthesis | 0.79 | ✓ Literature |
| 8 | Azathioprine | Purine synthesis | 0.77 | ✓ Standard care |
| 9 | Mycophenolate | Immune suppression | 0.75 | ✓ Literature |
| 10 | Baricitinib | JAK inhibition | 0.73 | ? Novel |

**Legend:**
- ✓ = Supported by existing evidence
- ? = Novel prediction (requires validation)

**Validation:**
```
Known drugs (Hit Rate): 7/10 = 70% ✓ Excellent!
Novel predictions: 3/10 = 30% (these are the exciting discoveries!)
```

### **8.3. Mechanistic Insights**

**Example: Why does the model predict Tocilizumab for Sjögren's?**

**Predicted Metapath:**
```
Tocilizumab --[targets]--> IL-6 --[upregulated_in]--> B-cells
                                        ↓
                                Autoantibody production
                                        ↓
                                Sjögren's Syndrome
```
```

Supporting Evidence:
```
1. BRG shows: IL-6 is central in Sjögren's pathology
2. Attention weights: Drug→Cytokine→Disease path has high importance
3. Training data: Tocilizumab works for RA and SLE (both involve IL-6)
4. Transfer: IL-6 dysregulation is common across autoimmune diseases
```

Interpretability:
The model doesn't just say "Tocilizumab might work" - it explains **WHY** through:
- Specific gene targets (IL-6)
- Biological pathways (B-cell activation)
- Shared mechanisms (autoantibody production)

This interpretability is **crucial for clinical adoption**!

---

9. Validation Strategy

We use a **5-layer validation framework** to ensure predictions are robust:

Layer 1: RepoDB Validation

What is RepoDB?
- Gold standard database of drug-disease pairs
- Contains both approved and failed indications
- ~6,000 drug-disease associations

How We Use It:
```
1. Hide RepoDB from training
2. Make predictions
3. Check: Did we rediscover known associations?

Metric: AUPRC, Hit@K
Target: AUPRC ≥ 0.92 (match SOTA)
```

Why This Matters:
If we can rediscover known drugs, our novel predictions are more trustworthy.

Layer 2: Literature Mining

Approach:
```
For each predicted drug-disease pair:
  1. Search PubMed for: "[Drug Name] AND [Disease Name]"
  2. Check for:
     - Clinical trials
     - Case reports
     - Mechanistic studies
     - Review articles
  3. Score evidence strength:
     - Strong: Clinical trial shows efficacy
```

```
   - Moderate: Case reports show promise
   - Weak: Mechanistic rationale only
   - None: No literature support
```

Target:
- 60%+ of top-50 predictions have literature support
- 40%+ have moderate/strong evidence

Example:
```
Prediction: Rituximab for Sjögren's
PubMed Search: "rituximab AND sjögren's syndrome"
Results: 127 papers
  - 15 clinical trials ✓
  - 8 case series ✓
  - 45 mechanistic studies ✓
Evidence: STRONG
```

Layer 3: Clinical Trial Evidence

Approach:
```
Search ClinicalTrials.gov for predicted drug-disease pairs

Categories:
  1. Completed trials (strong evidence)
  2. Active trials (moderate evidence)
  3. Planned trials (weak evidence)
  4. No trials (novel prediction)
```

Target:
- 30%+ of top-50 predictions have clinical trial evidence
- 20%+ are truly novel (no trials)

Example:
```
Prediction: Tocilizumab for Sjögren's
ClinicalTrials.gov: NCT01782235
  Status: Completed Phase II trial
  Results: Modest improvement in symptoms
Evidence: STRONG
```

Layer 4: Pathway Analysis

Biological Plausibility Check:
```
For each prediction:
  1. Extract predicted metapaths
  2. Map to KEGG/Reactome pathways
  3. Check if pathway is dysregulated in disease
  4. Verify drug targets pathway components
```

```
Score:
  - Highly plausible: Direct pathway match
  - Plausible: Indirect pathway connection
  - Speculative: Weak pathway link
```

Target:
- 80%+ of predictions are biologically plausible
- 50%+ have direct pathway evidence

Example:
```
Prediction: Baricitinib for Sjögren's
Pathway: JAK-STAT signaling
  - KEGG: hsa04630
  - Dysregulated in Sjögren's: ✓ YES
  - Baricitinib targets: JAK1/JAK2 ✓
  - Downstream: IFN pathway ✓ (key in Sjögren's)
Score: HIGHLY PLAUSIBLE
```

Layer 5: Expert Review

Process:
```
1. Select top 20 predictions
2. Prepare evidence packages:
   - Predicted mechanism
   - Literature support
   - Pathway analysis
   - Attention visualizations
3. Submit to rheumatology experts
4. Gather feedback:
   - Clinical plausibility
   - Safety concerns
   - Trial feasibility
```

Target:
- 70%+ of top-20 deemed "clinically plausible"
- At least 5 recommendations for "worth investigating further"

Why This Matters:
Expert validation is crucial for translating predictions to clinical practice.

9.1. Validation Scorecard

For Each Predicted Drug, We Report:
```
Drug: Tocilizumab
Disease: Sjögren's Syndrome
Confidence: 0.85

Validation:
```

  [✓] Layer 1: RepoDB confirmed (Phase II trial data)
  [✓] Layer 2: 15+ PubMed papers, 3 clinical trials
  [✓] Layer 3: Active Phase III trial (NCT01782235)
  [✓] Layer 4: IL-6 pathway strongly implicated
  [✓] Layer 5: Expert rating: "Highly plausible"

Overall: STRONG CANDIDATE (5/5 layers passed)
Recommendation: Prioritize for further investigation
```

---

## **10. Team & Timeline**

### **10.1. Team Structure**

**Team Size:** 4 members

**Yatin (Team Lead - You):**
- **Strengths:** Strong technical skills, architecture design, complex algorithms
- **Responsibilities:**
  - Model architecture implementation (NBFNet, BioPathNet++)
  - Core algorithm development
  - Complex debugging
  - Technical leadership
- **Time Commitment:** 40-50 hrs/week
- **Key Deliverables:**
  - NBFNet implementation
  - BioPathNet++ architecture
  - Training pipeline

**Srijan (Strong Technical):**
- **Strengths:** Data processing, PyTorch, validation frameworks
- **Responsibilities:**
  - Data pipeline development
  - Training infrastructure
  - Transfer learning experiments
  - Validation framework
- **Time Commitment:** 30-40 hrs/week
- **Key Deliverables:**
  - Data loaders
  - Training loop
  - Validation scripts

**Charan (Moderate Technical):**
- **Strengths:** Data analysis, visualization, documentation
- **Responsibilities:**
  - Dataset organization
  - Visualization creation
  - Documentation writing
  - Validation support
- **Time Commitment:** 25-30 hrs/week
- **Key Deliverables:**
  - Data statistics
  - Result visualizations
```

  - Progress reports

Praveen (Learning):
- **Strengths:** Eager to learn, documentation, testing
- **Responsibilities:**
  - Data download/verification
  - Basic validation tasks
  - Documentation
  - Testing scripts
- **Time Commitment:** 20-25 hrs/week
- **Key Deliverables:**
  - Data verification reports
  - Test scripts
  - Documentation

10.2. Project Timeline

Total Duration: 15 weeks (Jan 30 - May 15, 2026)

Major Milestones:
```
Week 0:  Jan 30 - Project kickoff, plan finalized
Week 3:  Feb 23 - Phase 1 complete ✓ (Data ready)
Week 9:  Apr 6  - Phase 2 complete (Model trained)
Week 15: May 15 - Final presentation
```

Review Schedule:
```
Phase 1 Review #1: Feb 3  - Dataset download plan
Phase 1 Review #2: Feb 10 - Data processing progress ← TOMORROW!
Phase 2 Review #1: Mar 2  - Model architecture
Phase 2 Review #2: Apr 20 - Training results
Final Presentation: May 15 - Complete results
```

10.3. Risk Management

High-Risk Items:

1. Model Training Challenges
```
Risk: BioPathNet++ training doesn't converge
Impact: High (core deliverable)
Mitigation:
  - Start with simpler baseline (TransE, GAT)
  - Incremental complexity (add NBFNet, then metapaths, then BRG)
  - Have fallback: Use pre-trained embeddings only
Status: Prepared
```

2. GPU Resource Constraints
```
Risk: Insufficient GPU memory for large graph
Impact: Medium (slows training)
Mitigation:
```

  - Use lab GPUs (priority access)
  - Implement mini-batching
  - Reduce batch size if needed
  - Google Colab Pro backup ($10/month)
Status: Mitigated
```

**3. Transfer Learning Failures**
```
Risk: Zero-shot doesn't work for Sjögren's
Impact: Medium (can fall back to few-shot)
Mitigation:
  - Try few-shot learning (5-10 examples)
  - Use data augmentation
  - Adjust model architecture
Status: Contingency planned
```

**4. Validation Gaps**
```
Risk: Predictions don't validate against RepoDB
Impact: High (credibility issue)
Mitigation:
  - Multiple validation layers (5-layer framework)
  - Conservative interpretation of results
  - Focus on mechanistic insights even if predictions imperfect
Status: Robust validation plan
```

### **10.4. Success Criteria**

**Minimum Success (Must Achieve):**
- ✓ Working end-to-end model
- ✓ Basic predictions for Sjögren's
- ✓ At least 3 validation layers implemented
- ✓ AUPRC ≥ 0.75 on RepoDB

**Target Success (Expected):**
- ✓ AUPRC ≥ 0.85 on RepoDB
- ✓ Transfer learning working (60%+ hit rate)
- ✓ All 5 validation layers complete
- ✓ Interpretable predictions with attention weights

**Stretch Goals (If Time Permits):**
- ✓ AUPRC ≥ 0.92 (match/beat SOTA)
- ✓ 70%+ literature validation
- ✓ Expert review completed
- ✓ Publication-ready results

---

## **11. Why This Project is Important**

### **11.1. For Patients**
```

Real Impact:
```
Current: Sjögren's patients have limited treatment options
         Often just symptom management (artificial tears, saliva substitutes)
         No disease-modifying drugs approved specifically for Sjögren's

With Our Model: Identify existing drugs that might work
                Faster access to treatment (drugs already approved)
                Lower cost (no new drug development needed)
                Potentially life-changing for 4 million+ patients worldwide
```

Example:
```
If we successfully predict that Rituximab (already FDA-approved for RA)
works for Sjögren's, doctors could:
  1. Prescribe it off-label immediately
  2. Start clinical trials faster (Phase II instead of Phase I)
  3. Get FDA approval in 2-3 years instead of 10-15 years
```

11.2. For Science

Advancing AI in Healthcare:

1. Transfer Learning in Biomedicine
```
First application of zero-shot transfer learning for rare disease drug repurposing
Demonstrates that AI can generalize across related diseases
Opens door for AI-assisted drug discovery for all rare diseases (7,000+)
```

2. Interpretable AI
```
Not a black box - we can explain WHY drugs are predicted
Attention weights show which biological pathways matter
Critical for clinical adoption (doctors need to understand reasoning)
```

3. Knowledge Graph Medicine
```
Shows how to integrate diverse biomedical data (DRKG + Orphanet + HPO)
Path-based reasoning captures complex biological mechanisms
Scalable to other disease areas
```

11.3. For Your Career

Skills Developed:

- Advanced deep learning (GNNs, transfer learning)
- Biomedical data science
- Large-scale data processing
- Scientific research methodology

- Technical communication

Potential Outcomes:

- Strong capstone project for portfolio
- Possible publication at AI/bioinformatics conference
- Industry-relevant experience (pharma AI roles)
- Foundation for Master's/PhD research

---

12. Summary

What We're Doing: Using AI to discover existing drugs that could treat rare autoimmune diseases, focusing on Sjögren's Syndrome.

How We're Doing It:

1. Build enriched knowledge graph (1.9M relationships)
2. Develop BioPathNet++ model (path-based GNN)
3. Train on common diseases (RA, SLE, MS)
4. Transfer knowledge to rare diseases (Sjögren's)
5. Validate predictions with 5-layer framework

Why It Matters:

- Faster, cheaper treatment discovery
- Help patients with rare diseases
- Advance AI in healthcare
- Demonstrate transfer learning in biomedicine

Current Status:

- Phase 1: ✅ Complete (data ready)
- Phase 2: 🔄 Starting (model implementation)
- Timeline: On track for May 15 completion

Expected Impact:

- 10+ novel drug candidates for Sjögren's
- AUPRC ≥ 0.85 (competitive with SOTA)
- 60-70% validation rate
- Potential clinical impact for millions of patients